NILS HARTMANN
https://nilshartmann.net



GraphQL

a critical review

API CONFERENCE 2025 | BERLIN, OCTOBER, 22. | HTTPS://GRAPHQL.SCHULE

NILS HARTMANN

nils@nilshartmann.net

Software Developer, Architect and Coach from Hamburg

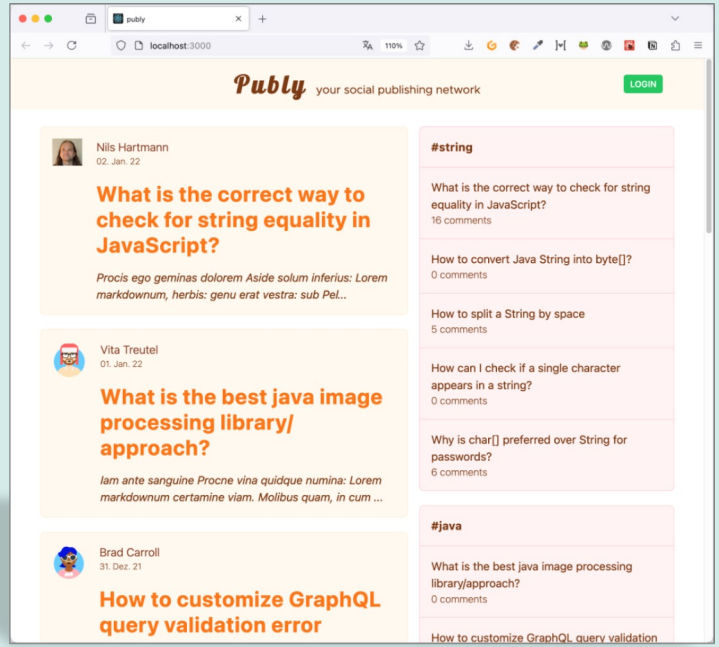
Java, Spring, GraphQL, TypeScript, React



https://graphql.schule/video-tutorial



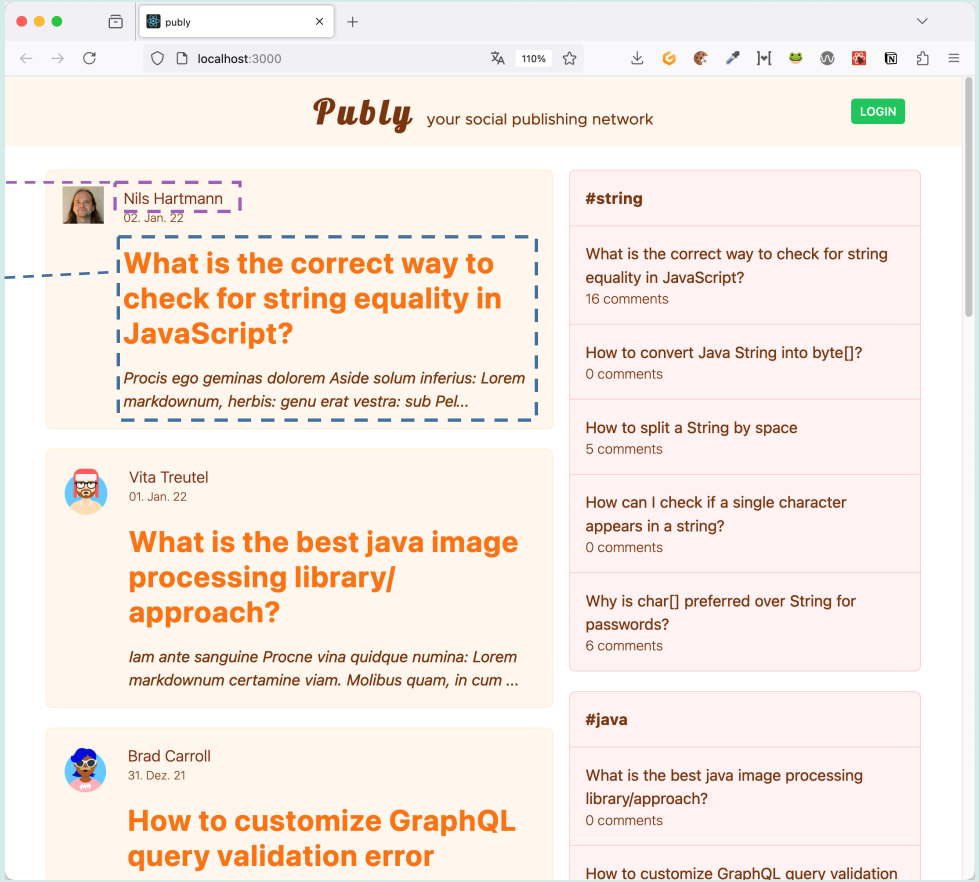
https://reactbuch.de



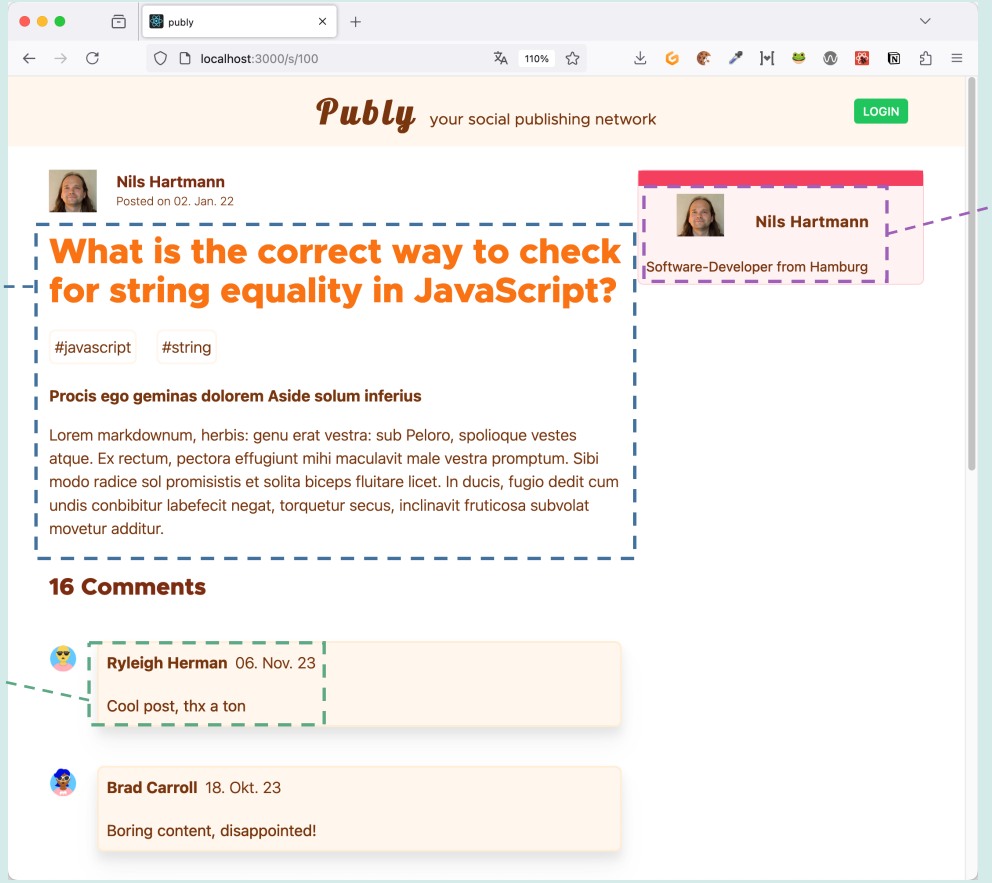
One application, 3 views...

User
- username

Story
- title
- excerpt



VIEW #1



Story
- title
- tags
- body

User
- username
- bio
- location

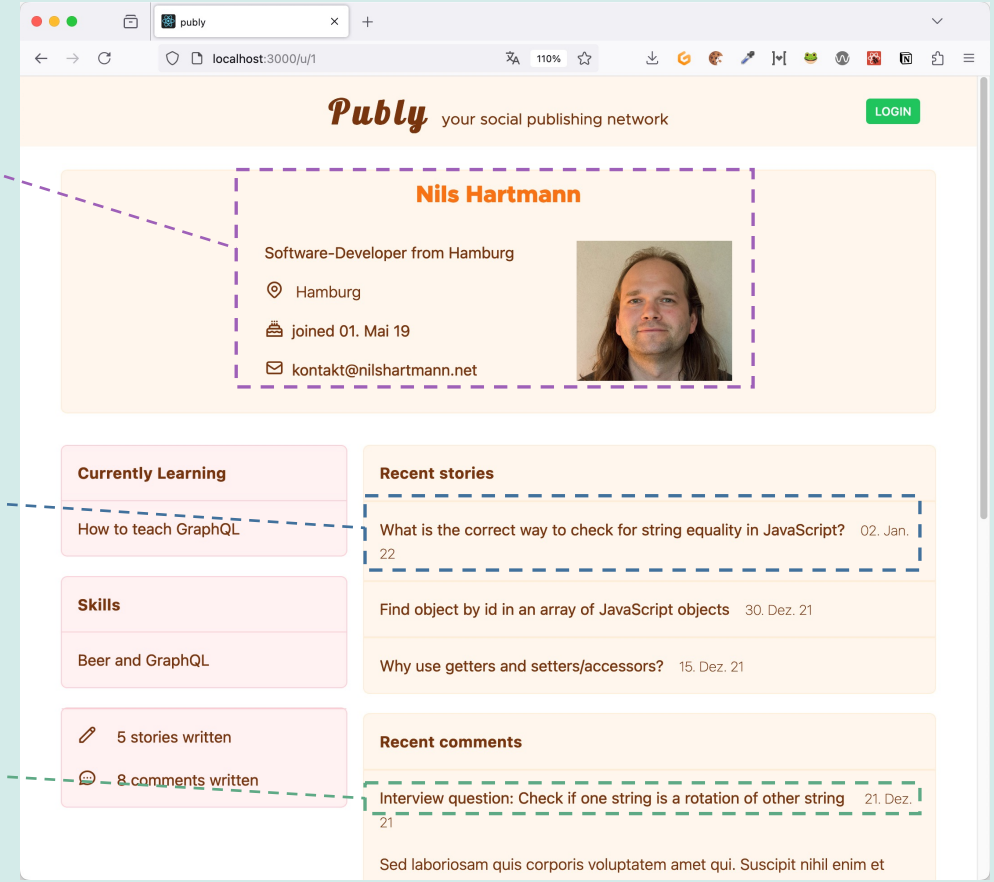
Comment
- writtenBy
- content

VIEW #2

User
- username
- bio
- location
- joined
- contact

Story
- title

Comment
- content



VIEW #3

An API for this example App …

AN API FOR THE EXAMPLE APP

Approach 1: Backend defines the API / data

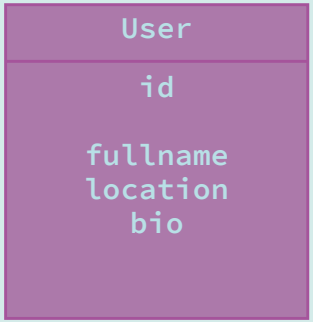
/api/story



/api/comment



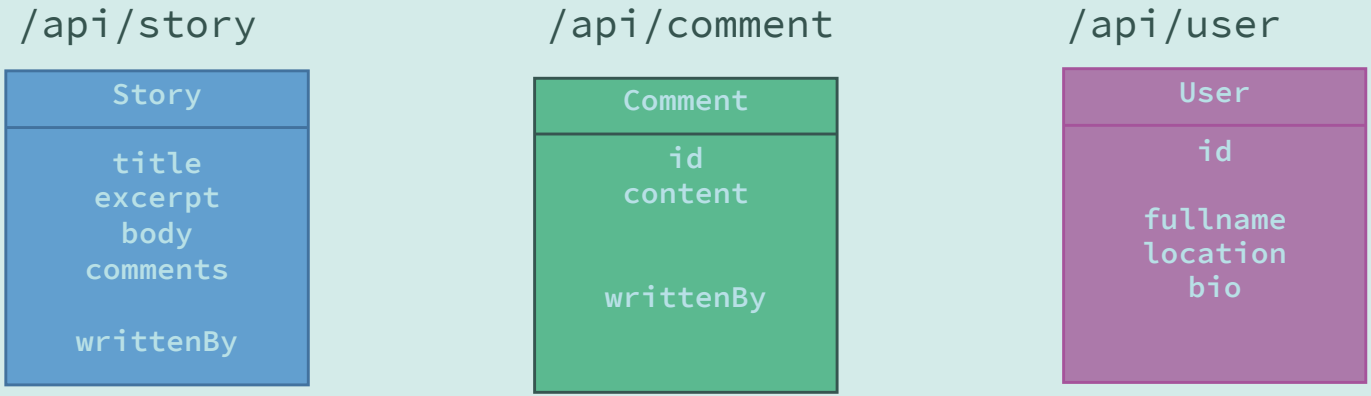
/api/user



AN API FOR THE EXAMPLE APP

Approach 1: Backend defines the API / data

REST / HTTP APIs



AN API FOR THE EXAMPLE APP

Approach 2: Client defines the API based on its requirements, views, use-cases, ...



AN API FOR THE EXAMPLE APP

Approach 2: Client defines the API based on its requirements, views, use-cases, ...

Backend for Frontend (BfF)



Approach 3: GraphQL...

Approach 3: GraphQL...

As approach 1: Server defines the data model



Approach 3: GraphQL...

As approach 1: Server defines the data model

...but the client can select in every request the data it wants to read from the model



Approach 3: GraphQL...

As approach 1: Server defines the data model

...but the client can select in every request the data it wants to read from the model

```
{ stories { title excerpt { user { fullname } } } }
```



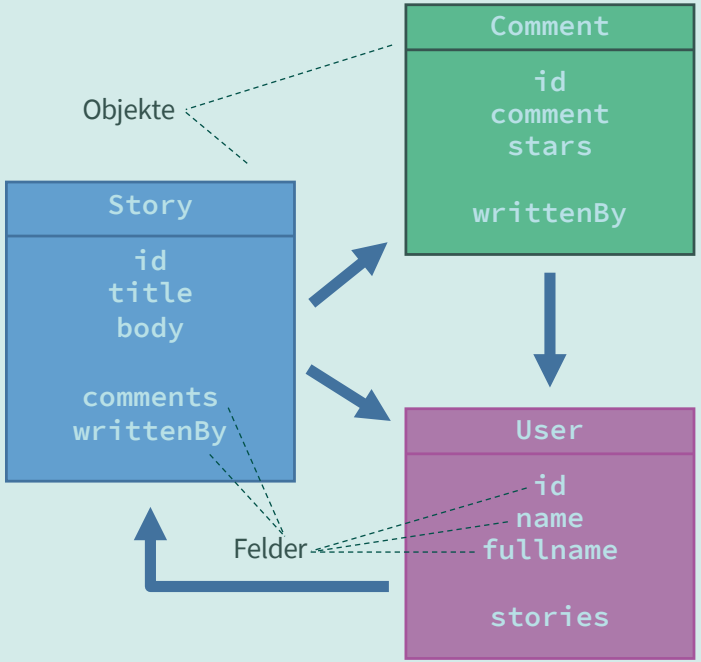
*"GraphQL is a **query language for APIs** and a **runtime for fulfilling those queries** with your existing data"*

- https://graphql.org

GraphQL

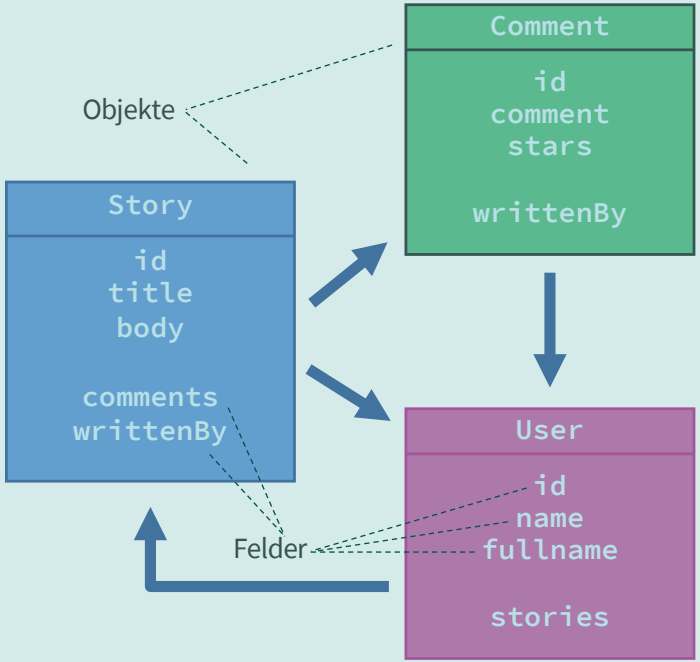
GRAPHQL SCHEMA

A GraphQL API provides a graph of *Objects* with *Fields*



A GraphQL API provides a graph of *Objects* with *Fields*

- The API is described in a **Schema**



GRAPHQL SCHEMA

A GraphQL API provides a graph of *Objects* with *Fields*

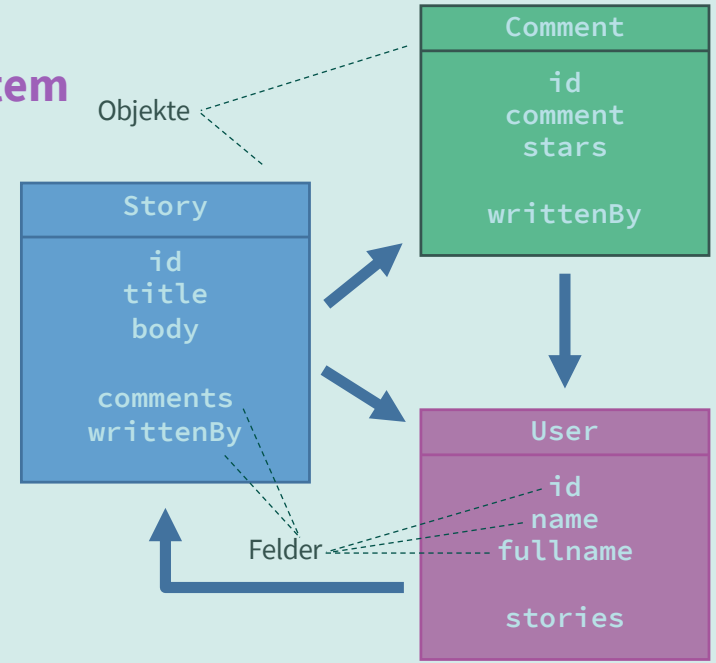
- The API is described in a **Schema**
- To make the API typesafe, it has an own **Typesystem**

```
type Story {
  id: ID!
  title: String!
  body: String!

  comments: [Comment!]
  writtenBy: User
}

type Story { ... }

type User { ... }
```



GRAPHQL SCHEMA

A GraphQL API provides a graph of *Objects* with *Fields*

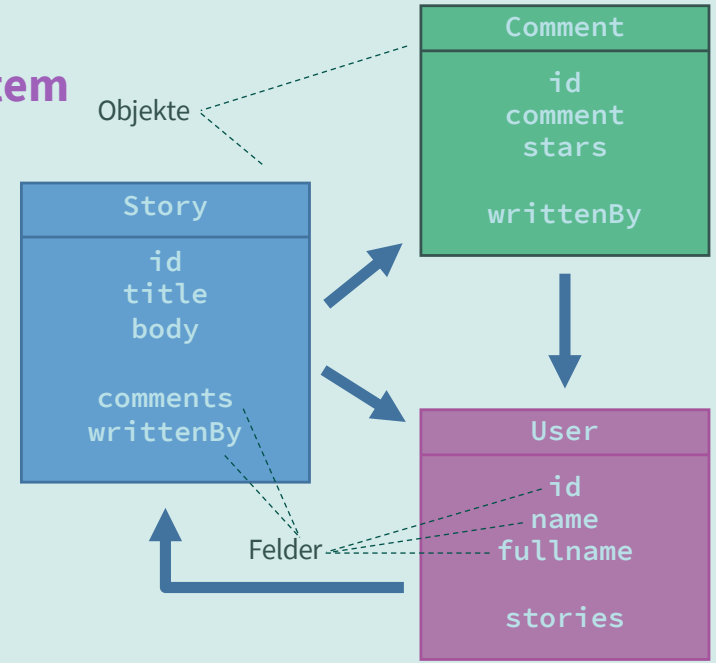
- The API is described in a **Schema**
- To make the API typesafe, it has an own **Typesystem**

```
type Story {
  id: ID!
  title: String!
  body: String!

  comments: [Comment!]
  writtenBy: User
}

type Story { ... }

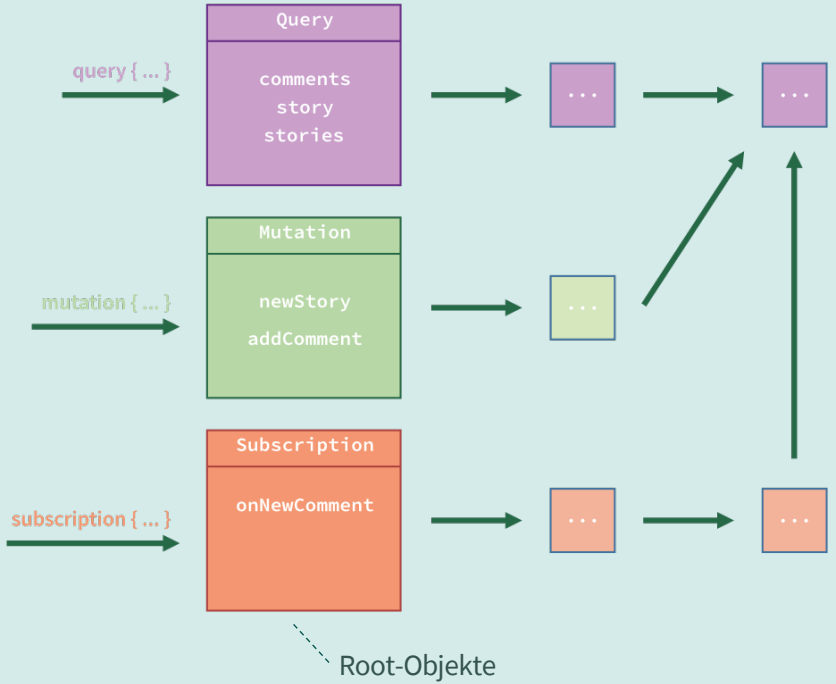
type User { ... }
```



👉 ***You can think of a Field as a Function that returns a value***

A GraphQL API provides a graph of *Objects* with *Fields*

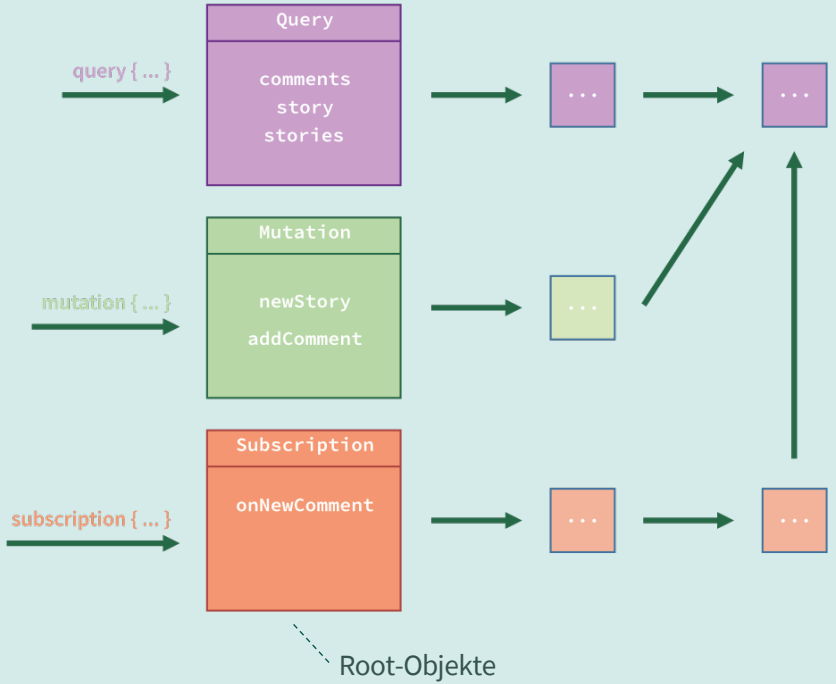
- The Graph starts at a ***Root*** *Object*



GRAPHQL SCHEMA

A GraphQL API provides a graph of *Objects* with *Fields*

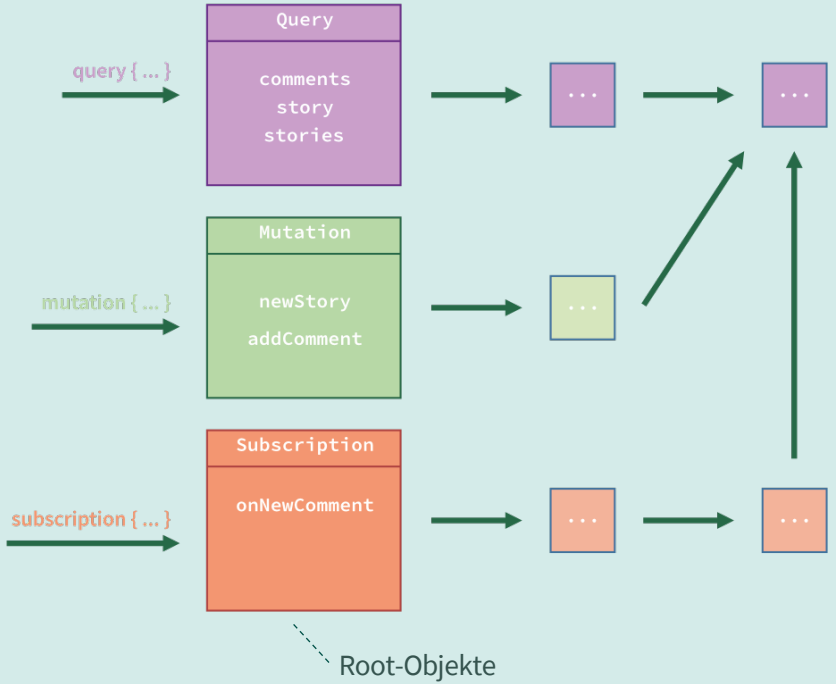
- The Graph starts at a ***Root*** *Object*
- There are up to three different Root objects: ***query***, ***mutation*** *und* ***subscription***



GRAPHQL SCHEMA

A GraphQL API provides a graph of *Objects* with *Fields*

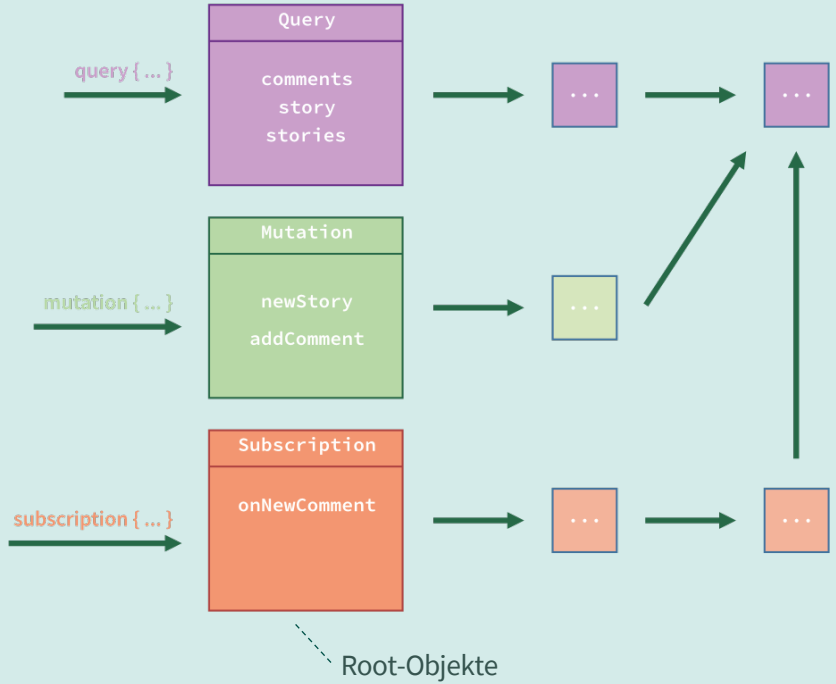
- The Graph starts at a ***Root*** *Object*
- There are up to three different Root objects: ***query***, ***mutation*** *und* ***subscription***
- Each Root object defines an *Entrypoint* in to the Graph (and thus your API)



GRAPHQL SCHEMA

A GraphQL API provides a graph of *Objects* with *Fields*

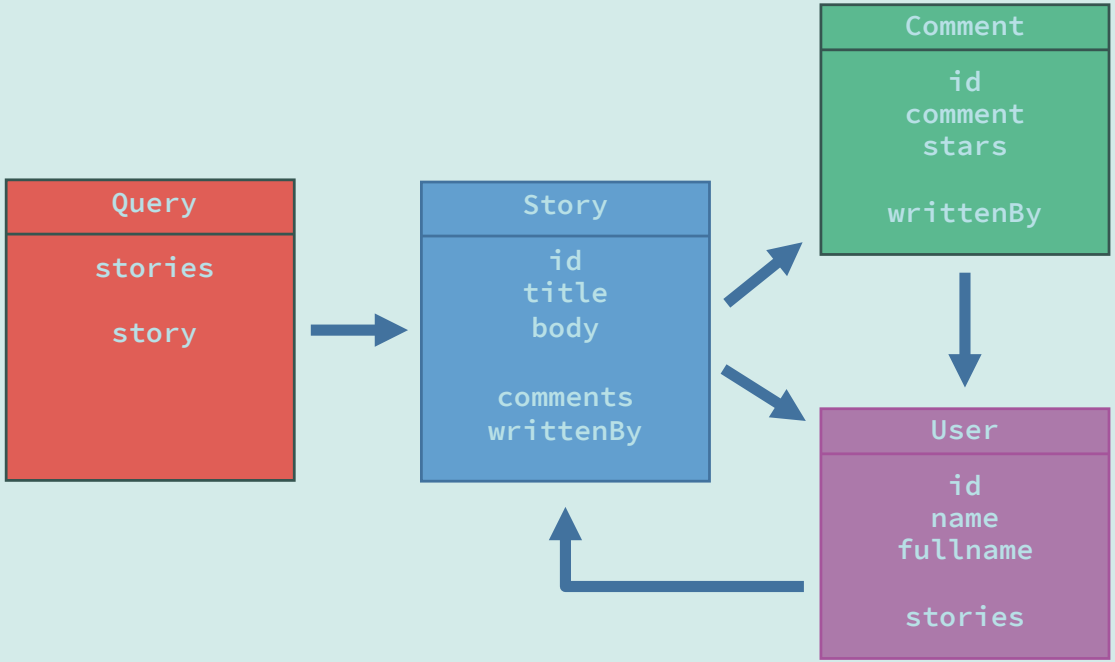
- The Graph starts at a ***Root*** *Object*
- There are up to three different Root objects: ***query***, ***mutation*** *und* ***subscription***
- Each Root object defines an *Entrypoint* in to the Graph (and thus your API)
- A ***Query*** has to start at one of the Root objects



The Query Language

QUERY LANGUAGE

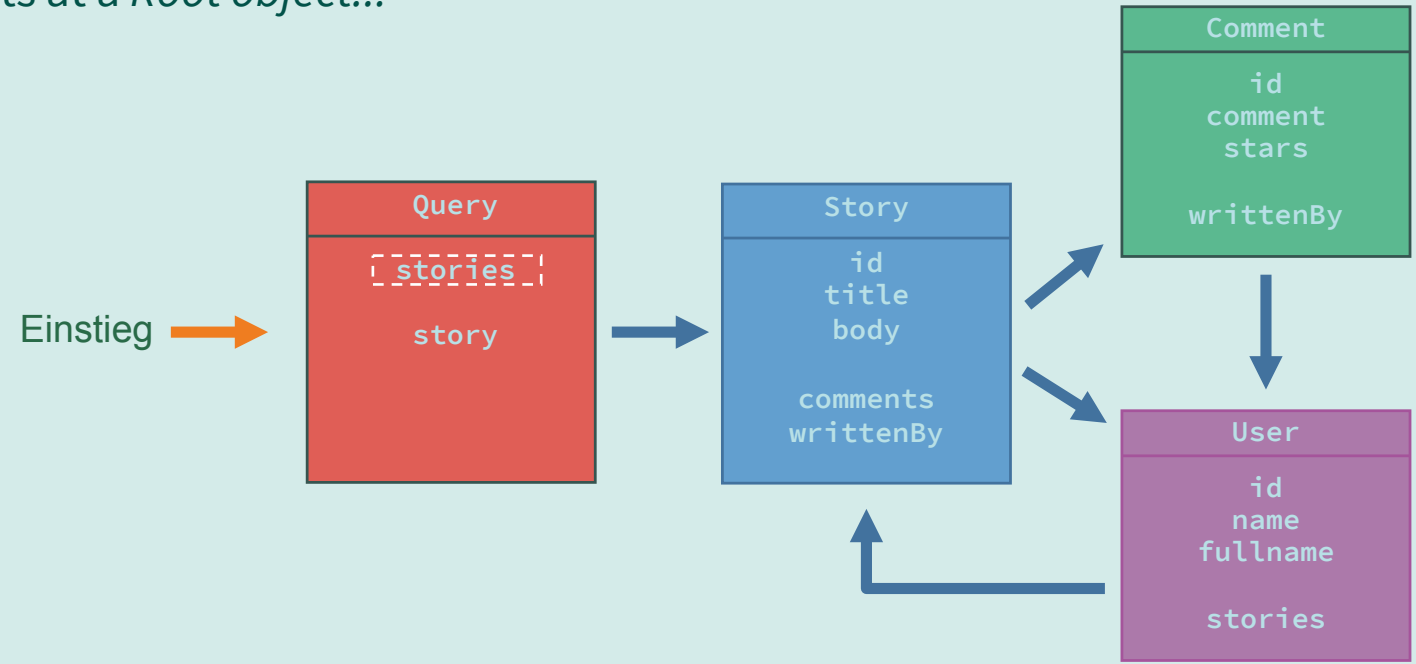
A client uses the *Query Language* to select *Fields* from the graph




```
query {                                                    }
```

A client uses the *Query Language* to select *Fields* from the graph

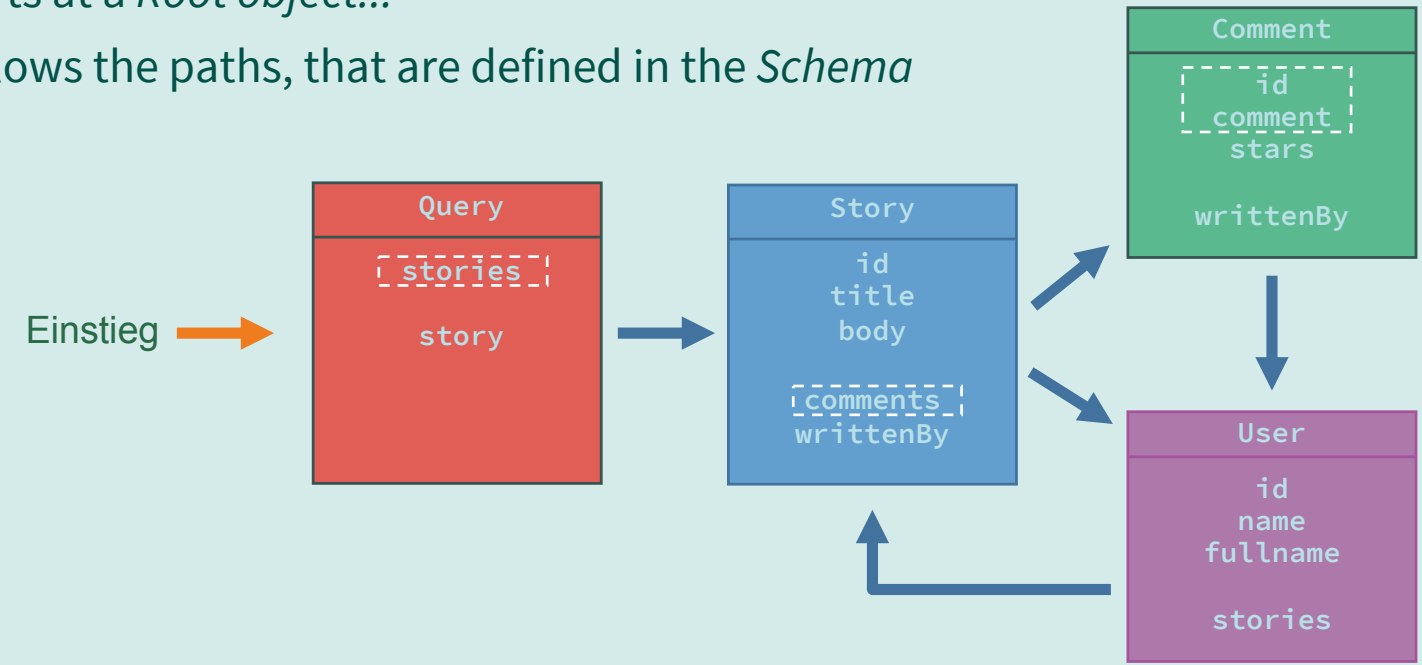
- The Query starts at a *Root object*...




```
query { stories                                    }
```

A client uses the *Query Language* to select *Fields* from the graph

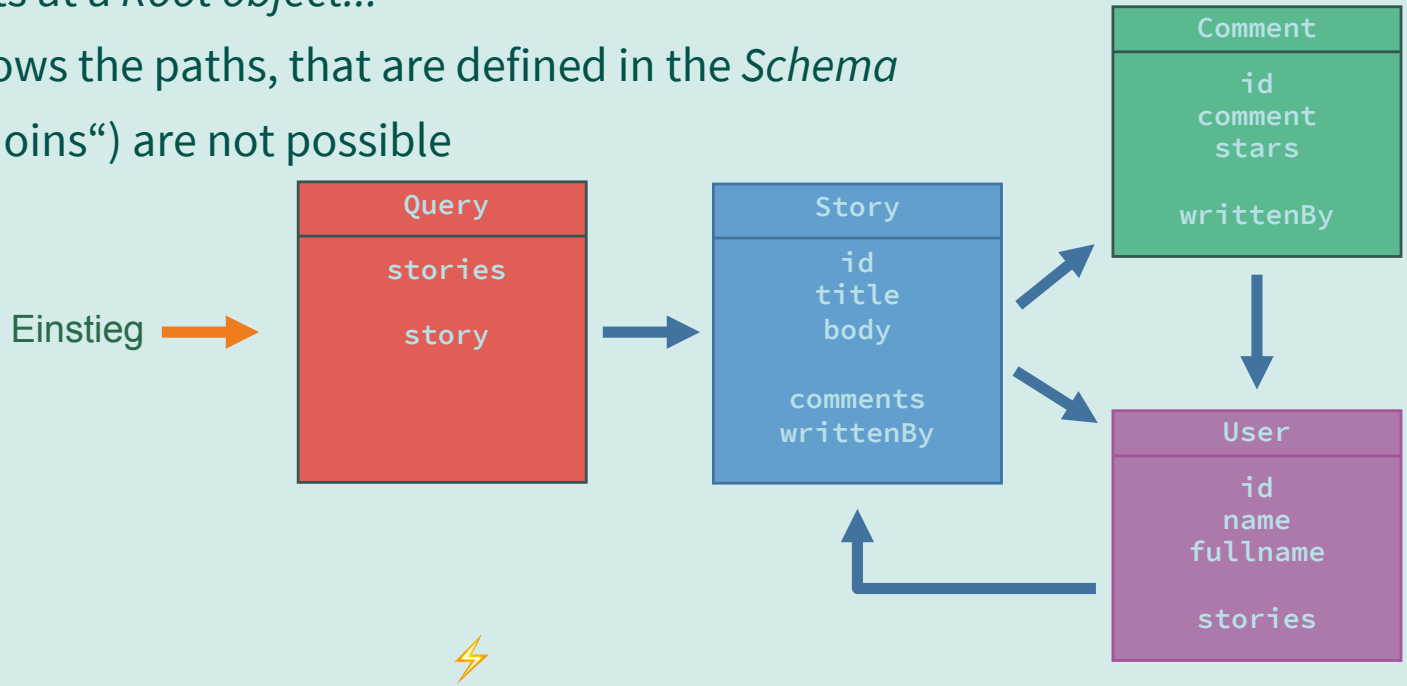
- The Query starts at a *Root object*...
- ...and then follows the paths, that are defined in the *Schema*



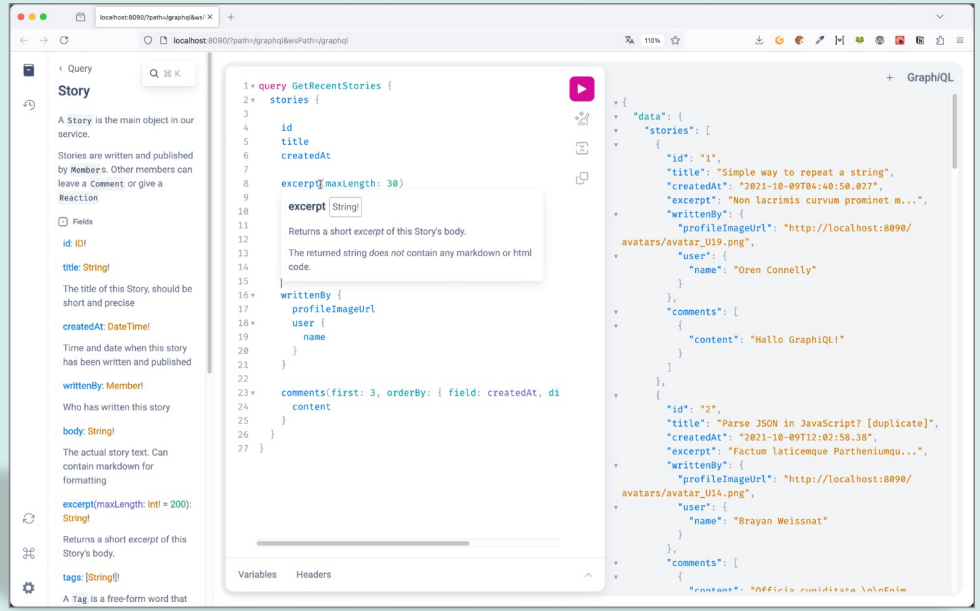

```
query { stories { comments { id comment } } }
```

A client uses the *Query Language* to select *Fields* from the graph

- The Query starts at a *Root object*...
- ...and then follows the paths, that are defined in the *Schema*
- Other paths („Joins“) are not possible




```
query { comments { id } }
```



- Ein Query
- Response
- Dokumentation
- Netzwerkverkehr (evtl.)

Demo: Query-Language

http://127.0.0.1:8080

Implementing

a GraphQL API

Implementing GraphQL APIs

- Usually you have to implement the „data fechting“ logic yourself
- There are frameworks for almost all programming languages

IMPLEMENTATION

The spec outlines how GraphQL Server should behave

- Almost all frameworks work somehow identically

Request processing workflow (simplified)

1. Server recieves HTTP Request with a Document containing an Operation

Request processing workflow (simplified)

1. Server recieves HTTP Request with a Document containing an Operation
2. Server validates the Document (Syntax and Schema)

Request processing workflow (simplified)

1. Server recieves HTTP Request with a Document containing an Operation
2. Server validates the Document (Syntax and Schema)
3. For all fields in the operation, a Resolver Functionen is invoked

Request processing workflow (simplified)

1. Server recieves HTTP Request with a Document containing an Operation
2. Server validates the Document (Syntax and Schema)
3. For all fields in the operation, a Resolver Functionen is invoked
4. **The Resolver Function is responsible for getting the Data for the field**

Request processing workflow (simplified)

1. Server recieves HTTP Request with a Document containing an Operation
2. Server validates the Document (Syntax and Schema)
3. For all fields in the operation, a Resolver Functionen is invoked
4. **The Resolver Function is responsible for getting the Data for the field**
5. The framework validates the collected data

Request processing workflow (simplified)

1. Server recieves HTTP Request with a Document containing an Operation
2. Server validates the Document (Syntax and Schema)
3. For all fields in the operation, a Resolver Functionen is invoked
4. **The Resolver Function is responsible for getting the Data for the field**
5. The framework validates the collected data
6. The response is sent back to the client

Request processing workflow (simplified)

1. Server recieves HTTP Request with a Document containing an Operation
2. Server validates the Document (Syntax and Schema)
3. For all fields in the operation, a Resolver Functionen is invoked
4. **The Resolver Function is responsible for getting the Data for the field**
5. The framework validates the collected data
6. The response is sent back to the client

👉 **Implementing the Resolver Functionen is our task**

RESOLVER FUNCTIONS

Resolver functions determine the value for a requested field

👉 A resolver function is a regular Function in your programming language

RESOLVER FUNCTIONS

Resolver functions determine the value for a requested field

👉 A resolver function is a regular Function in your programming language

👉 It can can do what ever it needs to do to get the requested data
(Read database, call other services, compute data)

RESOLVER FUNCTIONS

Resolver functions determine the value for a requested field

- 👉 A resolver function is a regular Function in your programming language
- 👉 It can can do what ever it needs to do to get the requested data
  (Read database, call other services, compute data)
- 👉 Think of a Controller method in REST APIs
- 👉 Think of a requested GraphQL field as a remote method invocation

IMPLEMENTATION

Example

Java with Spring Boot

```
query {
  story(storyId: "1")
  {
    writtenBy
    {
      name
    }
  }
}
```

```
@Controller
public class StoryController {

    private final StoryRepository storyRepository;
    private final UserMicroService userMicroService;

    public StoryController(StoryRepository storyRepository,
                           UserMicroService userMicroService) {...}


    @QueryMapping
    public Story story(@Argument String storyId) {
        return storyRepository.findById(storyId);
    }

    @SchemaMapping
    public User writtenBy(Story story) {
        return userMicroService.fetchUser(story.getWrittenBy().getUserId());
    }
}
```

Claims

about

GraphQL

CLAIMS ABOUT GRAPHQL

"GraphQL APIs are more complex to develop then REST APIs"

"GraphQL APIs are more complex to develop then REST APIs"

REST APIs are somehow “simple": Each endpoint returns *its own* Ressource

- The returned resource is already known at *Development time*

"GraphQL APIs are more complex to develop then REST APIs"

REST APIs are somehow “simple": Each endpoint returns *its own* Ressource

- The returned resource is already known at *Development time*

GraphQL kann return multiple objects

- What needs to be returned is only known at *Runtime*

CLAIMS ABOUT GRAPHQL

"GraphQL APIs are more complex to develop then REST APIs"

REST APIs are somehow “simple": Each endpoint returns *its own* Ressource

- The returned resource is already known at *Development time*

GraphQL kann return multiple objects

- What needs to be returned is only known at *Runtime*

👉 **Optimizations** (performance for example) are more complicated as in REST APIs

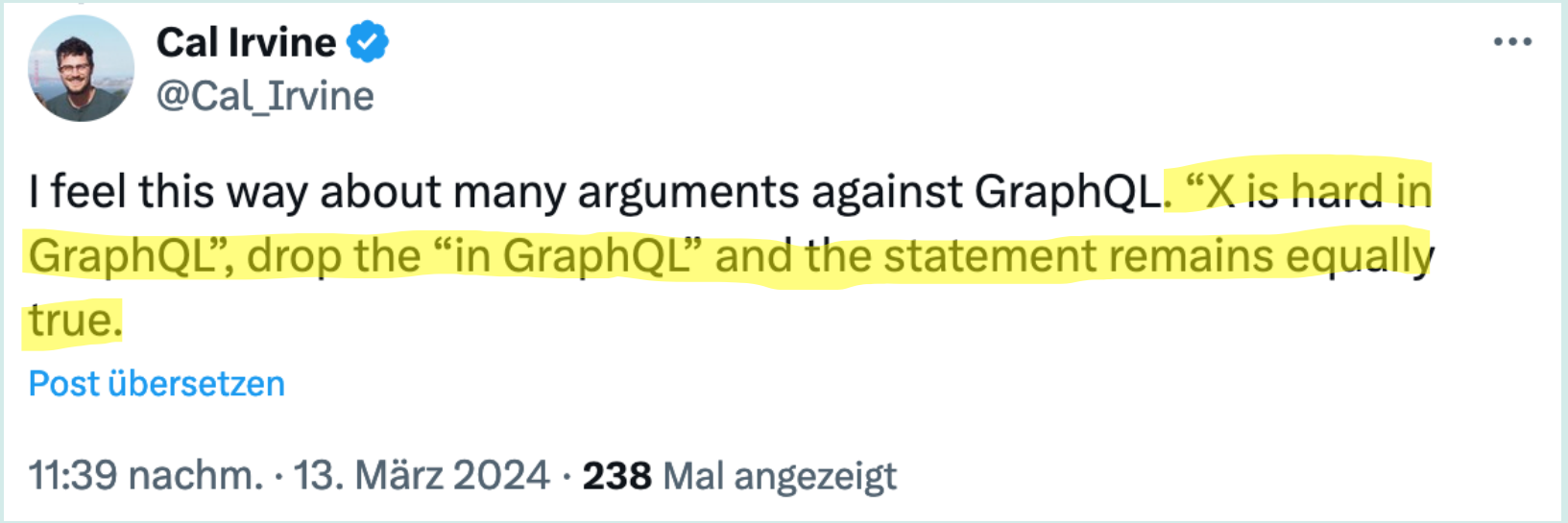
"GraphQL APIs are more complex to develop then REST APIs"

Complexity of developlemnt depends on various criterias:

- Size and complexity of your schema (domain!)
- Number and type of datasources
- Complexity and structure of your input data
    - do they fit the schema more or less?
    - Legacy data?

"GraphQL APIs are more complex to develop then REST APIs"

Summary



https://x.com/Cal_Irvine/status/1768044262124323175

CLAIMS ON GRAPHQL

"GraphQL is SQL, but for the Frontend"

That‘s wrong

"GraphQL is SQL, but for the Frontend"

That‘s wrong

- SQL is way more powerful than the GraphQL language
  - We do not have ORDER BY, LIMIT, JOIN
  - We do not have „generic“ queries

"GraphQL is SQL, but for the Frontend"

That‘s wrong

- SQL is way more powerful than the GraphQL language
  - We do not have ORDER BY, LIMIT, JOIN
  - We do not have „generic“ queries
- GraphQL does not return whole databases back to the client
- Data must not even contained in a database

"GraphQL is SQL, but for the Frontend"

That‘s wrong

- SQL is way more powerful than the GraphQL language
  - We do not have ORDER BY, LIMIT, JOIN
  - We do not have „generic“ queries
- GraphQL does not return whole databases back to the client
- Data must not even contained in a database
- Data and its structure is determined by our schema and our resolver functions

CLAIMS ON GRAPHQL

"GraphQL is for lazy people that do not like to design APIs"

That is wrong

CLAIMS ON GRAPHQL

"GraphQL is for lazy people that do not like to design APIs"

That is wrong

- Suggests that with graphql all your data is exposed unstructured to the client

CLAIMS ON GRAPHQL

"GraphQL is for lazy people that do not like to design APIs"

That is wrong

- Suggests that with graphql all your data is exposed unstructured to the client
- But schema design is as important as with other technologies

CLAIMS ON GRAPHQL

"GraphQL is for lazy people that do not like to design APIs"

That is wrong

- Suggests that with graphql all your data is exposed unstructured to the client
- But schema design is as important as with other technologies
- How our API looks like and how good or bad it fits its needs is up to us

CLAIMS ON GRAPHQL

"GraphQL is for lazy people that do not like to design APIs"

That is wrong

- Suggests that with graphql all your data is exposed unstructured to the client
- But schema design is as important as with other technologies
- How our API looks like and how good or bad it fits its needs is up to us
- You can take care… or not… as with all technologies

CLAIMS ON GRAPHQL

“Caching with GraphQL is impossible"

That‘s… difficult

- Claim normally refers to GraphQL requests over HTTP

“Caching with GraphQL is impossible"

Problems with caching #1

- HTTP Post Requests not cacheable
    - The new spec „GraphQL over HTTP“ describes GET as request method

“Caching with GraphQL is impossible"

Problems with Caching #2

- When a client requests **multiple objects**, the shorted cache time must „win“

“Caching with GraphQL is impossible"

Problems with Caching #2

- When a client requests **multiple objects**, the shorted cache time must „win“
- Example: Story and User
  - User is probably more or less "stable", good candidate for caching
  - Story probably not, cause things can change (Likes, corrections, …)

“Caching with GraphQL is impossible"

Problems with Caching #2

- When a client requests **multiple objects**, the shorted cache time must „win“
- Example: Story and User
  - User is probably more or less "stable", good candidate for caching
  - Story probably not, cause things can change (Likes, corrections, …)
- In **REST** each Resource can be cached individually

"GraphQL saves data (traffic) compared to REST APIs"

Only partly correct

- Correct is: network requests and thus latency can be better with a one-fit‘s-it-all-Query

"GraphQL saves data (traffic) compared to REST APIs"

Only partly correct

- Correct is: network requests and thus latency can be better with a one-fit‘s-it-all-Query
- But: data will not de-duplicated or normalized
    - A query of X Stories with the same publisher returns X publishers (even when they‘re them same for some stories)
    - Can be more efficent then with REST API but does not have to

"GraphQL saves data (traffic) compared to REST APIs"

Only partly correct

- Correct is: network requests and thus latency can be better with a one-fit‘s-it-all-Query
- But: data will not de-duplicated or normalized
    - A query of X Stories with the same publisher returns X publishers (even when they‘re them same for some stories)
    - Can be more efficent then with REST API but does not have to
- An "unmodified“ response in REST does not return any data at all

CLAIMS ON GRAPHQL

“I can do the same with REST..."

Offen correct

“I can do the same with REST..."

Offen correct

- With search params its possible to limit requested attributes of a resource

CLAIMS ON GRAPHQL

“I can do the same with REST..."

Offen correct

- With search params its possible to limit requested attributes of a resource
- A schema can be defined with OpenAPI/Swagger

“I can do the same with REST..."

Offen correct

- With search params its possible to limit requested attributes of a resource
- A schema can be defined with OpenAPI/Swagger
- With REST it‘s possible to return „sub-objects“

“I can do the same with REST..."

Offen correct

- With search params its possible to limit requested attributes of a resource
- A schema can be defined with OpenAPI/Swagger
- With REST it‘s possible to return „sub-objects“
- With HATEOS a server can guide a client to the related data of a resource

“I can do the same with REST..."

Offen correct

- With search params its possible to limit requested attributes of a resource
- A schema can be defined with OpenAPI/Swagger
- With REST it‘s possible to return „sub-objects“
- With HATEOS a server can guide a client to the related data of a resource

- But:
  - all that is built into GraphQL, you get everything from one hand

CLAIMS ON GRAPHQL

“Why should I then use GraphQL?"

There are good reasons:

“Why should I then use GraphQL?"

There are good reasons:

- Over-/Underfetching (overhyped in my opinion)

“Why should I then use GraphQL?"

There are good reasons:

- Over-/Underfetching (overhyped in my opinion)
- Esp. in frontends you often have the need to query data in various forms

“Why should I then use GraphQL?"

There are good reasons:

- Over-/Underfetching (overhyped in my opinion)
- Esp. in frontends you often have the need to query data in various forms
- Schema evolution allows steady improvements of the API

CLAIMS ON GRAPHQL

"Why should I then use GraphQL?"

There are good reasons:

- Over-/Underfetching (overhyped in my opinion)
- Esp. in frontends you often have the need to query data in various forms
- Schema evolution allows steady improvements of the API
- Tools and typesafety have very good developer experience

CLAIMS ON GRAPHQL

"Why should I then use GraphQL?"

There are good reasons:

- Over-/Underfetching (overhyped in my opinion)
- Esp. in frontends you often have the need to query data in various forms
- Schema evolution allows steady improvements of the API
- Tools and typesafety have very good developer experience
- APIs are good self explorable and discoverable

“Why should I then use GraphQL?"

There are good reasons:

- Over-/Underfetching (overhyped in my opinion)
- Esp. in frontends you often have the need to query data in various forms
- Schema evolution allows steady improvements of the API
- Tools and typesafety have very good developer experience
- APIs are good self explorable and discoverable
- Error handling more explizit as in REST

“Why should I then use GraphQL?"

There are some reasons against it:

“Why should I then use GraphQL?"

There are some reasons against it:

- REST is mainstream, well understood and widely used

“Why should I then use GraphQL?"

There are some reasons against it:

- REST is mainstream, well understood and widely used
- Consumers and developers know how REST works

“Why should I then use GraphQL?"

There are some reasons against it:

- REST is mainstream, well understood and widely used
- Consumers and developers know how REST works
- Caching on HTTP level is easy

“When should I use GraphQL?"

it depends...

(as always 😇)

NILS HARTMANN
https://nilshartmann.net

Thank you!

Slides: https://graphql.schule/api-conference

Questions and contact: nils@nilshartmann.net

My GraphQL Workshops: https://graphql.schule

